Pilas: Estructura de dato en la cual el acceso está *limitado* al elemento más recientemente insertado. Posee una estructura *LIFO (Last In, First Out)*.
Operaciones naturales: { insertar, eliminar, buscar } || { apilar, desapilar, cima }

- Apilar: agrega un elemento al tope de la pila
- Desapilar: elimina un elemento del tope de la pila

Pilas - Implementación en estructuras estáticas

Las pilas se pueden implementar guardando los elementos en un *array de longitud fija*, utilizando una variable para saber la posición del último elemento guardado (tope).

- Si un programa intenta sacar un elemento de una *pila vacía* se producirá un error por *desbordamiento negativo (**underflow**)*.
- Lo mismo sucederá cuando intente poner un elemento en una *pila llena*, solamente que ahora se producirá un *desbordamiento positivo (**overflow**) o rebosamiento*.

Algoritmo de Inserción de elementos || { Apilar }

1. Verificar si la pila no está llena.
2. Incrementar en 1 el apuntador (cima) de la pila.
3. Almacenar el elemento en la posición del apuntador de la pila.

Algoritmo de Eliminación de elementos || { Desapilar }

1. Verificar que la pila no esté vacía.
2. Leer el elemento de la posición del apuntador (cima) de la pila.
3. Decrementar en 1 el apuntador de la pila.

[Tener en cuenta que si la pila por ejemplo está vacía, el apuntador será = -1, mientras que si está llena el apuntador será el último índice que le corresponde al arreglo **]**

Aplicación de las Pilas

- Compiladores
- Sistemas Operativos
- Programas de aplicaciones

Colas: Estructura de dato en la cual se eliminan desde los primeros elementos en ingresar y se ingresan a partir del último elemento del arreglo. El acceso está igual de *limitado* que para pilas. Tiene una estructura *FIFO (First In, First Out)*.

Se debe determinar un *inicio* y un *final*.

Operaciones naturales:

- Insertar (al final) || Encolar
- Eliminar (al principio) || Desencolar

Colas - Implementación en Estructuras Estáticas.

Las colas, al igual que las pilas, se deben implementar guardando los elementos en un *array de longitud fija*. En este caso debemos utilizar *dos marcadores* para mantener las posiciones al *frente* y *final* de la cola.

- Un marcador almacena la posición de la cabeza (frente) de la cola. Sería el *primer elemento no vacío del arreglo*.
- El otro almacena el *primer espacio vacío que le sigue al último elemento* de la cola.
- Al igual que en pilas, una cola puede estar *vacía o llena*.

Algoritmo de Inserción de elementos || { Encolar }

1. Verificar que no esté la cola llena,
2. Insertar el elemento en la posición final.
3. Aumentar en 1 el final.

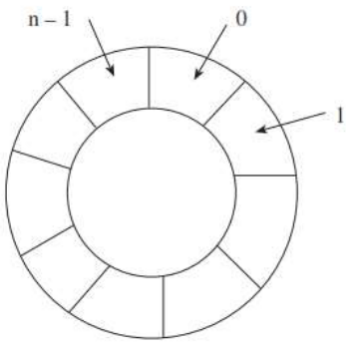
Algoritmo de Eliminación de elementos || { Desencolar }

1. Verificar que no esté la cola vacía.
2. Aumento en 1 el frente.

[En la eliminación solamente puedo borrar hasta la *posición del final* y no hasta el último índice del arreglo como puede ser intuitivo en una primer instancia]

Colas circulares en Array

- Es la forma más eficiente de almacenar una cola en un array.
- En las *colas circulares* se une el extremo final con el extremo inicial.



- De esta manera, la totalidad de las posiciones del array se utilizan para poder almacenar elementos de la cola *sin necesidad de desplazar elementos*.
- Para poder implementar esto podemos usar la *teoría de los restos* para generar índices *entre 0 y el máximo tamaño de la cola* $final = (final + 1)\%TAM;$ || $frente = (frente + 1)\%TAM;$

Aplicación de las colas

- Colas de mensajes
- Colas de tareas a realizar por una impresora
- Colas de prioridades